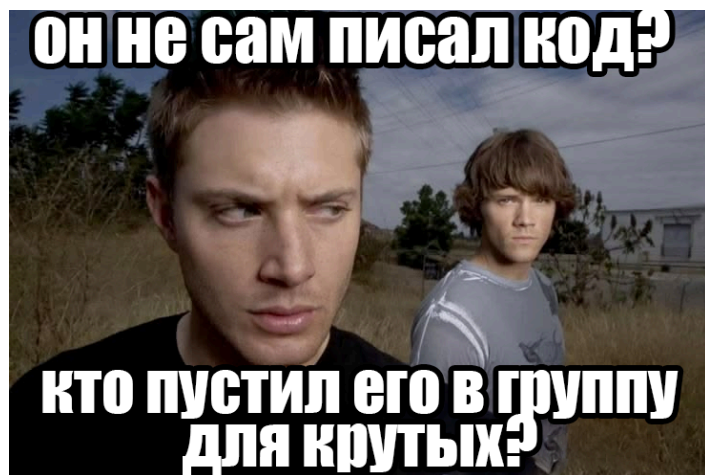


Лабораторная работа №18

Использование RecyclerView: адаптер, ViewHolder, оптимизация списка

Основные правила

1. Не бойтесь задавать любые вопросы
2. Если хотите сделать что-то свое, то предлагайте – будем думать
3. Советую читать текст, тут написано много полезного
- 4.



О чем лабораторная?

На прошлом занятии вы познакомились с RecyclerView и научились создавать адаптер. Сегодня применим эти знания на практике: создадим список с условным отображением и обработкой клика.

Главное, что нужно понять

- **ViewHolder** - объект, который хранит ссылки на View внутри элемента списка. Благодаря ему не нужно каждый раз вызывать `findViewById()`

Часть 1. Самостоятельная работа

Задание

Выберите одну из тем или используйте свою:

1. **Фильмы** - название, год, рейтинг. Рейтинг ≥ 8 - зеленый цвет текста
2. **Книги** - название, автор, год. Книги старше 2000 года - жирный шрифт
3. **Товары** - название, цена, категория. Товары со скидкой - зачёркнутая цена
4. **Музыка** - исполнитель, альбом, год. Альбомы после 2010 - зелёный фон
5. **Своя тема** - любой список с условным отображением (как в темах выше)

Требования к любой теме (*если что-то забыли нажмите сюда*):

- Создайте модель данных с полями для условного отображения
- Создайте макет элемента списка
- Создайте ViewHolder
- Создайте адаптер (конструктор, `getItemCount()` и другие методы - напишите самостоятельно)
- Добавьте условную логику отображения (цвет, шрифт, видимость элементов)
- Добавьте обработку клика на элемент списка (Toast с данными элемента)

Подсказки

Для условного отображения используйте `if` внутри `onBindViewHolder`:

```
if (movie.getRating() >= 8.0) {
    holder.textRating.setTextColor(Color.GREEN);
} else {
    holder.textRating.setTextColor(Color.BLACK);
}
```

Для обработки клика используйте `setOnClickListener`:

```
holder.itemView.setOnClickListener(v -> {
    Toast.makeText(v.getContext(), movie.getTitle(), Toast.LENGTH_SHORT).show();
});
```

Часть 2. Зачем нужен ViewHolder?

Объяснение

Когда вы прокручиваете список, элементы, которые уходят за пределы экрана, не уничтожаются. Вместо этого RecyclerView **переиспользует** их для отображения новых данных.

Без ViewHolder вам пришлось бы каждый раз вызывать `findViewById()` - это медленная операция. ViewHolder сохраняет ссылки на View, поэтому `findViewById()` вызывается только один раз - при создании ViewHolder.

Эксперимент

Попробуйте убрать ViewHolder и вызывать `findViewById()` прямо в `onBindViewHolder()`:

```
@Override
public void onBindViewHolder(BookViewHolder holder, int position) {
    Book book = books.get(position);
    // Так делать не нужно:
    TextView title = holder.itemView.findViewById(R.id.text_title);
    TextView author = holder.itemView.findViewById(R.id.text_author);
    title.setText(book.getTitle());
```

```
author.setText(book.getAuthor());  
}
```

Запустите приложение и быстро прокрутите список. Разница в производительности будет заметна на большом списке

Требования к итоговой версии

1. Приложение отображает список с помощью RecyclerView
2. Адаптер наследуется от `RecyclerView.Adapter`
3. Используется отдельный класс ViewHolder
4. Реализована условная логика отображения (в зависимости от темы)
5. Реализована обработка клика на элемент списка (Toast)
6. Все текстовые данные вынесены в `strings.xml`